# Lab2 BombLab 实验报告

## 各关的关键汇编、计算关系与推理过程

（以下关键汇编并非 `objdump` 的逐行原文， `fail` 、 `loop` 、 `found` 、 `left` 、 `right` 、 `default` 、 `length_ok` 、 `done` 是人为定义的跳转标签，便于说明，反汇编里实际显示为地址。）

## 第一关

关键汇编：

```
phase_01:
  mov    %rsi,%r12
  mov    %rdi,%rbx
  call   valid_u32_fields
  ...
  lea    -0x14(%rbp),%rdx
  mov    $0x403189,%esi
  mov    %rbx,%rdi
  call   __isoc99_sscanf
  cmp    $0x1,%eax
  jne    fail
  mov    %r12,%rdi
  call   p1_compute
  cmp    %eax,-0x14(%rbp)
  sete   %al

p1_compute:
  mov    %rdi,%rdx
  mov    %rdi,%rax
  shr    $0x20,%rax
  mov    %rdx,%rsi
  mov    %eax,%edi
  shr    $0x10,%rsi
  call   p1_step1
  mov    %edi,%esi
  mov    %edx,%edi
  mov    %eax,%ecx
  call   p1_step0
  mov    %ecx,%esi
  mov    %eax,%edi
```

```
  call   p1_step2

p1_step1:
  lea    (%rdi,%rdi,2),%eax
  add    %esi,%eax
  xor    $0x84c6,%eax

p1_step0:
  xor    $0x1be7,%esi
  lea    (%rsi,%rdi,2),%eax

p1_step2:
  sub    %esi,%edi
  lea    0x4fc7(%rdi,%rdi,2),%eax
```

先分析 `phase_01` 的参数传递：该函数将 `rsi` 保存到 `r12`，`rdi` 保存到 `rbx`；随后 `rbx` 作为输入字符串传给 `sscanf`，`r12` 作为 `challenge` 传给 `p1_compute`。由此可知，第一个参数是输入行，第二个参数是 `challenge`。

`valid_u32_fields` 第二个参数为 1，说明本关只接受一个整数；格式串位于 `0x403189`，内容为 `"%u"`，解析结果存放在 `rbp-0x14`。`p1_compute` 的返回值与 `rbp-0x14` 比较，`sete` 将相等结果转换为 1。因此，本关的目标是使 `p1_compute(challenge)` 的输出等于输入整数。

`p1_compute` 先将 `challenge` 复制到 `rdx`，再将 `challenge` 右移 32 位放入 `rax`，因此 `eax` 为 `challenge` 的高 32 位；随后将 `challenge` 复制到 `rsi` 并右移 16 位，而 `p1_step1` 只使用 `esi`，所以取得的是 `challenge >> 16` 的低 32 位；`rdx` 的低 32 位即 `challenge` 的低 32 位。

虽然传入的是 64 位寄存器，但三个小函数内部只使用 32 位寄存器，高 32 位不参与计算，加、减、乘的结果按 (2^{32}) 取模。小函数的作用如下：

- `p1_step1` 使用 `lea (%rdi,%rdi,2),%eax` 计算 `3 * rdi`，加上 `esi` 后异或 `0x84c6`。
- `p1_step0` 先将 `esi` 异或 `0x1be7`，再使用 `lea (%rsi,%rdi,2),%eax` 计算 `rsi + 2*rdi`。其中 `rdi` 是低 32 位，`rsi` 是异或后的高 32 位。
- `p1_step2` 先执行 `edi = edi - esi`，再计算 `3 * edi + 0x4fc7`。

恢复出的计算关系：

```
h = challenge >> 32
m = challenge >> 16
```

```
l = challenge

r1 = (3*h + m) ^ 0x84c6
r2 = (h ^ 0x1be7) + 2*l
answer = 3*(r2 - r1) + 0x4fc7
```

## 第二关

关键汇编：

```
phase_02:
  mov    $0x6,%esi
  call   valid_u32_fields
  ...
  lea    -0x2c(%rbp),%rax
  lea    -0x3c(%rbp),%rcx
  push   %rax
  lea    -0x30(%rbp),%rax
  lea    -0x40(%rbp),%rdx
  push   %rax
  lea    -0x34(%rbp),%r9
  lea    -0x38(%rbp),%r8
  call   __isoc99_sscanf

  mov    -0x40(%rbp),%esi
  shr    $0x7,%rax
  movzbl %al,%eax
  add    $0x1f,%eax
  cmp    %eax,%esi

  mov    $0x1c,%ecx
  mov    $0x1,%edx

loop:
  lea    (%rsi,%rsi,2),%esi
  shr    %cl,%rax
  and    $0x1f,%eax
  add    %edx,%eax
  xor    %esi,%eax
  mov    -0x40(%rbp,%rdx,4),%esi
  cmp    %eax,%esi
  add    $0x1,%rdx
  add    $0x5,%ecx
```

```
  cmp    $0x6,%rdx
  jne    loop
```

`phase_02` 将 6 传给 `valid_u32_fields`，说明本关要求 6 个整数。`sscanf` 有 6 个输出指针，分别指向 `rbp-0x40`、`rbp-0x3c`、`rbp-0x38`、`rbp-0x34`、`rbp-0x30`、`rbp-0x2c`，每个相隔 4 字节。因此这 6 个整数在栈上构成连续数组，`-0x40(%rbp,%rdx,4)` 即 `v[rdx]`。

第一个整数单独检查：将 `challenge` 右移 7 位，`movzbl %al,%eax` 只保留最低 8 位，再加 `0x1f`。因此 `v[0]` 由 `challenge` 的第 7～14 位决定。

后五个整数在循环中计算：`ecx` 是移位量，初值为 `0x1c`，即 28（不是从第15位继续），每轮加 5，因此依次为 28、33、38、43、48；`and $0x1f` 表示取低 5 位。`edx` 是轮号 `i`，从 1 开始。循环体依次完成以下操作：

- `lea (%rsi,%rsi,2),%esi` 将上一个字段 `v[i-1]` 乘以 3；
- `shr %cl,%rax` 按 `cl` 指定的位数右移，`and $0x1f` 取出 5 位；
- `add %edx,%eax` 加上轮号 `i`；
- `xor %esi,%eax` 与 `3*v[i-1]` 异或；
- 最后与 `v[i]` 比较。

恢复出的计算关系：

```
v[0] = ((challenge >> 7) & 0xff) + 0x1f

i = 1..5:
    shift = 28 + 5*(i-1)
    v[i] = (((challenge >> shift) & 0x1f) + i) ^ (3 * v[i-1])
```

## 第三关

关键汇编：

```
phase_03:
  mov    $0x2,%esi
  call   valid_u32_fields
  ...
  mov    %r12d,%edi
  and    $0x7,%edi
  cmp    %edi,-0x18(%rbp)
  jne    fail
  mov    %r12,%rax
```

```
  mov    %r12,%rsi
  shr    $0x3,%rax
  shr    $0x20,%rsi
  xor    %eax,%esi
  call   p3_dispatch
  cmp    %eax,-0x14(%rbp)
  sete   %al

p3_dispatch:
  mov    %edi,%eax
  mov    %esi,%edi
  cmp    $0x7,%eax
  ja     default
  notrack jmp *0x405820(,%rax,8)
```

第三关要求两个整数，第一个输入先与 `challenge & 7` 比较，因此第一个整数实际是 `switch` 的 selector，由题目直接给出；第二个整数才是需要计算的答案。

`p3_dispatch` 的两个参数分别是 selector 和 `arg2`。selector 先被复制到 `eax`，`arg2` 被移到 `edi`，随后 `cmp $0x7,%eax` 检查范围，最后 `jmp *0x405820(,%rax,8)` 从跳转表中取第 `selector` 个 8 字节地址并跳转。`0x405820` 是一个代码地址数组，每个表项 8 字节，对应一个 selector。

`arg2` 的来源为：`rax = challenge >> 3`，`rsi = challenge >> 32`，`xor %eax,%esi` 将两者的低 32 位异或。由于 selector 占用最低 3 位，计算 `arg2` 时从第 3 位开始取值。

最初我按 case 名称直接对应，即认为 selector 0 进入 `p3_case_0`、selector 1 进入 `p3_case_1`，导致公开测试多数失败，仅少数 seed 碰巧通过。随后使用 `objdump -s -j .rodata --start-address=0x405820 ...` 打印 8 个表项，发现表项地址并非按 `case_0`、`case_1` ……排列；每个表项先跳到一小段中转代码，再由中转代码跳到真正的 case。逐个跟踪后得到真实映射：

| selector | 实际 case | 公式（`a = arg2`） |
|---|---|---|
| 0 | case_0 | `5*a + 0xdad1` |
| 1 | case_1 | `(a ^ 0x2f5f) + (a >> 1)` |
| 2 | case_3 | `(4*a) ^ 0xe2da` |
| 3 | case_7 | `5*a + 0x109dc` |
| 4 | case_6 | `(a ^ 0xc0ac) + (a >> 1)` |
| 5 | case_5 | `(2*a) ^ 0x33f4` |

| selector | 实际 case | 公式（`a = arg2`） |
| --- | --- | --- |
| 6 | case_2 | `5*a + 0x2f68e` |
| 7 | case_4 | `5*a + 0x2943` |

## 第四关

关键汇编：

```
phase_04:
  mov    %r12d,%edi
  xor    %esi,%esi
  mov    $0x7f,%edx
  shr    $0x20,%rax
  shr    $0x7,%rcx
  and    $0x7f,%edi
  xor    %eax,%ecx
  call   p4_walk
  cmp    %eax,-0x14(%rbp)
  sete   %al

p4_walk:
  mov    %edi,%eax
  mov    %edx,%edi
  sub    %esi,%edi
  shr    $1,%edi
  add    %esi,%edi
  cmp    %eax,%edi
  je     found

  push   %rbp
  mov    %rsp,%rbp
  push   %rbx
  lea    (%rdi,%rcx,1),%ebx
  cmp    %edi,%eax
  jb     left

right:
  lea    0x1(%rdi),%esi
  mov    %eax,%edi
  call   p4_walk
  lea    (%rax,%rax,4),%eax
  xor    %ebx,%eax
```

```
  xor    $0x1f,%eax
  ret

left:
  lea    -0x1(%rdi),%edx
  mov    %eax,%edi
  call   p4_walk
  lea    (%rax,%rax,4),%eax
  xor    %ebx,%eax
  xor    $0xe,%eax
  ret

found:
  lea    0x39(%rdi),%eax
  xor    %ecx,%eax
```

第四关只要求一个整数。`phase_04` 从 `challenge` 中取得三项：`target = challenge & 0x7f`，范围为 0～127；`low = 0`；`high = 0x7f`；`key` 为 `challenge >> 7` 与 `challenge >> 32` 的低 32 位异或。随后调用 `p4_walk(target, low, high, key)`。

`p4_walk` 实现二分查找，它将 `target` 存入 `eax`，然后计算 `mid`：`edi = high - low`，右移 1 位（等价于除以2），再加 `low`。接着比较 `mid` 与 `target`：若相等则跳到 `found`，返回 `(mid + 0x39) ^ key`；若不相等，则先将 `mid + key` 存入 `ebx`，再判断 `target` 与 `mid` 的大小。`cmp %edi,%eax` 后接 `jb left`，表示 `target < mid` 时进入左半边；未跳转则说明 `target > mid`，进入右半边。右半边将 `low` 改为 `mid+1`，左半边将 `high` 改为 `mid-1`，然后递归调用自身。

递归返回后还需进行回代：`lea (%rax,%rax,4),%eax` 将子结果乘以 5，`xor %ebx,%eax` 异或此前保存的 `mid + key`，最后右半边再异或 `0x1f`，左半边再异或 `0xe`。因此答案并非最底层的 `(mid+0x39)^key`，而是从最底层逐层乘 5、异或并叠加方向常数后得到的结果。

恢复出的计算关系：

```
target = challenge & 0x7f
key    = (challenge >> 7) ^ (challenge >> 32)

walk(target, low, high):
    mid = low + (high - low) / 2
    if mid == target: return (mid + 0x39) ^ key
    temp = mid + key
```

```
    if target > mid:
        return (5 * walk(target, mid+1, high) ^ temp) ^ 0x1f
    else:
        return (5 * walk(target, low, mid-1) ^ temp) ^ 0xe
```

# 第五关

关键汇编：

```
phase_05:
  lea    -0x14(%rbp),%rsi
  mov    %r12,%rdi
  call   p5_traverse
  mov    %eax,%edx
  cmp    %edx,-0x1c(%rbp)
  jne    fail
  mov    -0x14(%rbp),%eax
  cmp    %eax,-0x18(%rbp)
  sete   %al

p5_traverse:
  and    $0x7,%eax
  shr    $0x3,%rcx
  mov    0x405b60(,%rax,8),%rdx
  shr    $0x20,%rdi
  shr    $0x6,%rax
  mov    0x4(%rdx),%esi
  xor    %edi,%eax
  xor    (%rdx),%eax
  xor    $0xc65c,%eax
  mov    %esi,(%r8)
  add    %esi,%eax
  and    $0x3,%edi
  je     done

loop:
  mov    0x8(%rdx),%rdx
  mov    0x4(%rdx),%esi
  xor    (%rdx),%eax
  add    %esi,%eax
  mov    %esi,(%r8)
  cmp    %ecx,%edi
  jae    loop
```

这一关要求两个整数。`phase_05` 将第二个输出整数的地址 `rbp-0x14` 传给 `p5_traverse`，返回值位于 `eax`；随后先比较第一个输入与 `eax`，再比较第二个输入与 `rbp-0x14` 中的值。因此本关答案有两个：函数返回值，以及通过指针写回的第二个值。

`p5_traverse` 首先取 `challenge & 7` 作为下标，从 `0x405b60` 取出一个 8 字节指针。该地址是 8 个链表头指针的数组，即 `head[0]` ～ `head[7]`。随后将 `challenge >> 3` 的低 2 位作为步数，取值范围为 0～3。

结点布局可从访存偏移确定：`mov 0x4(%rdx),%esi` 读取偏移 4 的值，`xor (%rdx),%eax` 读取偏移 0 的值，`mov 0x8(%rdx),%rdx` 读取偏移 8 的指针作为下一个结点。因此结点布局为：

```
偏移 0：val1，4 字节
偏移 4：val2，4 字节
偏移 8：next，8 字节指针
```

整个结构体 16 字节。`0x405b60` 的每个表项是 8 字节指针，指向某个结点。

初始值的计算为：先将 `challenge >> 6` 与 `challenge >> 32` 的低 32 位异或，再异或头结点的 `val1`，异或 `0xc65c`，加头结点的 `val2`；同时将 `val2` 写入输出。若步数不为 0，则循环执行：`rdx = next`，读取新结点的 `val2`，异或其 `val1`，加其 `val2`，并更新输出。

循环次数的细节如下：`edi` 为步数，`ecx` 从 1 开始，每轮先加到 2 再比较 `edi` 与 `ecx`，`jae` 成立才继续。因此步数为 1 时执行一次，步数为 3 时执行三次，恰好等于 `(challenge >> 3) & 3`。

为确认链表内容，使用 gdb 查看 `0x405b60` 与各结点。按 `next` 指针顺序，12 个结点为：

```
(5126,1410) → (11198,2910) → (30717,2637) → (18454,2440)
→ (30373,1661) → (19463,1825) → (64621,1030) → (65508,583)
→ (55439,311) → (56070,2067) → (36190,9) → (59793,827)
```

发现 8 个 `head` 是同一条 12 结点链表的 8 个后缀，`head[i]` 正好指向第 `i` 个结点。实现时将 12 组 `val1`、`val2` 按上述链表顺序存成两个数组，因此下标加一等价于 `node = node->next`。

恢复出的计算关系：

```
idx   = challenge & 7
steps = (challenge >> 3) & 3
node  = head[idx]

x = (challenge >> 6) ^ (challenge >> 32)
x = x ^ node->val1 ^ 0xc65c
x = x + node->val2
second = node->val2

重复 steps 次:
    node = node->next
    x = x ^ node->val1
    x = x + node->val2
    second = node->val2

answer1 = x
answer2 = second
```

# 第六关

关键汇编:

```
phase_06:
  mov    %r12,%rdi
  call   p6_dispatch
  cmp    %eax,-0x14(%rbp)
  sete   %al

p6_dispatch:
  mov    %rdi,%rbx
  shr    $0x6,%rdi
  mov    %rbx,%rax
  shr    $0x20,%rax
  xor    %eax,%edi
  mov    %rbx,%rax
  and    $0x38,%ebx
  and    $0x7,%eax
  call   *0x405e00(,%rax,8)
  xor    $0xd985,%eax
  mov    %eax,%edi
  call   *0x405e00(%rbx)
  add    $0xd2be,%eax
```

这关只要一个整数，`phase_06` 将 `challenge` 传给 `p6_dispatch`，再比较返回值与输入。`p6_dispatch` 先从 `challenge` 中取得两个 3 位下标：`idx1 = challenge & 7`，`idx2 = (challenge >> 3) & 7`。同时计算第一次调用的参数：`challenge >> 6` 与 `challenge >> 32` 的低 32 位异或。

随后执行两次间接调用：第一次使用 `0x405e00(,%rax,8)`，其中 `rax` 为 `idx1`，即取函数表第 `idx1` 项；第一次的结果异或 `0xd985`，作为第二次调用的参数；第二次使用 `0x405e00(%rbx)`，其中 `rbx` 为 `challenge & 0x38`，即 `idx2 * 8`，取第 `idx2` 项；最后加 `0xd2be`。`0x405e00` 是函数指针数组，每个元素 8 字节。

函数表的 8 项对应关系为：

| idx | 函数 | 公式（`a` 是参数） |
|---|---|---|
| 0 | `p6_op_5` | `((a << 2) \| (a >> 30)) ^ 0xabe0` |
| 1 | `p6_op_4` | `(a + 0xb336) ^ (a >> 2)` |
| 2 | `p6_op_5` | 同上 |
| 3 | `p6_op_5` | 同上 |
| 4 | `p6_op_0` | `((a >> 7) ^ a) + 0x629c` |
| 5 | `p6_op_3` | `(8*a + 0xc23f) ^ (5*a)` |
| 6 | `p6_op_2` | `(a ^ 0xfbbc) + 2*a` |
| 7 | `p6_op_1` | `(a >> 7) \| (a << 25)` |

恢复出的计算关系：

```
idx1 = challenge & 7
idx2 = (challenge >> 3) & 7
arg1 = (challenge >> 6) ^ (challenge >> 32)

res1 = table[idx1](arg1)
res2 = table[idx2](res1 ^ 0xd985)
answer = res2 + 0xd2be
```

## Secret 关

关键汇编：

```
secret_gate:
  call   strlen
```

```
  sub    $0x24,%rax
  cmp    $0x7f,%rax
  jbe    length_ok
  ...

length_ok:
  call   secret_copy
  cmpl   $0x4e45504f,-0xb0(%rbp)
  jne    fail
  lea    -0xac(%rbp),%rdi
  call   secret_phase

secret_phase:
  mov    $0x3,%esi
  call   valid_u32_fields
  ...
  mov    %r13d,%ecx
  and    $0x3,%ecx
  cmp    %ecx,-0x2c(%rbp)
  jne    fail
  mov    %r12,%rsi
  imul   $0xf84,%ecx,%r9d
  mov    %r13,%r8
  shr    $0x20,%rsi
  shr    $0x20,%r8
  xor    %r12d,%esi
  xor    %r13d,%r8d
  shr    $0x10,%r13
  mov    %esi,%edi
  xor    %r13d,%r9d
  xor    %r8d,%edi
  call   secret_mix
  xor    %r9d,%eax
  cmp    %eax,-0x28(%rbp)
  jne    fail
  shr    $1,%esi
  lea    0xb432(%r8,%rcx,1),%edx
  add    %esi,%edx
  xor    %edx,%eax
  cmp    -0x24(%rbp),%eax
```

`secret_gate` 首先检查整行长度。`strlen` 的返回值减 36，再与 127 进行无符号比较，`jbe` 成立才继续。因此行长必须在 36～163 之间；若行长小于 36，减 36 会回

绕成较大的数，比较失败。该检查表明第七行至少需要 36 个字符，且不能过长。

随后将整行复制到从 `rbp-0xd0` 开始的缓冲区，并比较 `rbp-0xb0` 处的 4 个字节与 `0x4e45504f`。`rbp-0xb0` 比 `rbp-0xd0` 大 `0x20`，即偏移 32；`0x4e45504f` 按小端存放为 `4f 50 45 4e`，即 `O P E N`。因此检查的是偏移 32 处的 `"OPEN"`，而非行首。

最初将 `OPEN` 放在行首，测试未通过。后在 gdb 中于 `0x40277b` 观察，发现 `rbp-0xd0` 开头确为 `OPEN`，但比较的是 `rbp-0xb0`，由此确认前面需要 32 字节填充。填充内容不参与检查，因此使用了 32 个 `'A'` 占位。

之后 `secret_phase` 从 `rbp-0xac` 开始解析，即从 `OPEN` 后面的空格开始。它先调用 `valid_u32_fields(...,3)`，再用 `sscanf` 读取 3 个整数，因此第七行格式为：

```
[32 个填充字节]OPEN [整数1] [整数2] [整数3]
```

`secret_phase` 中第一个整数必须等于 `context & 3`；随后分别拆分 `challenge` 与 `context`：

- `esi` 来自 `challenge`：`challenge >> 32` 的低 32 位异或 `challenge` 的低 32 位；
- `r8` 来自 `context`：`context >> 32` 的低 32 位异或 `context` 的低 32 位；
- `r9` 为 `v0 * 0xf84` 再异或 `context >> 16` 的低 32 位。

接着 `edi = esi ^ r8`，调用 `secret_mix`，返回值再异或 `r9`，得到第二个整数。`secret_mix` 执行循环左移 1 位、加 `0xf84`、异或 `0xb432`。第三个整数由第二个整数、`r8`、`v0` 与 `esi >> 1` 再异或得到。

`context` 由六关答案计算：`transcript_context` 逐行扫描每个十进制整数，将阶段号、字段号与数值混入一个 64 位数，因此必须前面六关均正确才能通过。

恢复出的计算关系：

```
v0       = context & 3
chal_xor = (challenge >> 32) ^ challenge
ctx_xor  = (context >> 32) ^ context
r9       = (v0 * 0xf84) ^ (context >> 16)

v1 = secret_mix(chal_xor ^ ctx_xor) ^ r9
v2 = v1 ^ (0xb432 + ctx_xor + v0 + (chal_xor >> 1))
```

## 本地自测结果截图

（验证前需先执行 `make clean && make`，确保 `defuser` 与当前 `defuser.c` 一致！一开始总是忘记编译，还以为全写错了。）

`python3 public_test.py`（前面六关的完整输出结果太长了没截进来）：

```
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

公开列表之外的 seed 验证（用的 123456789）：

```
lenovo_unix@SYLVIA-LAPTOP:~/ics-2026-bomblab$ ./defuser --seed 123456789 --phase 1 | ./bomb --seed 123456789 --phase 1
```

```
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

--------------------------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x7a91dd183971ee2e
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

```
lenovo_unix@SYLVIA-LAPTOP:~/ics-2026-bomblab$ ./defuser --seed 123456789 --phase 2 | ./bomb --seed 123456789 --phase 2
```

```
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

--------------------------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x310e0831409afde5
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

```
lenovo_unix@SYLVIA-LAPTOP:~/ics-2026-bomblab$ ./defuser --seed 123456789 --phase 3 | ./bomb --seed 123456789 --phase 3
```

```
BOMBLAB 2026                                        这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

--------------------------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x851e061616a5bee5
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]
```

```
lenovo_unix@SYLVIA-LAPTOP:~/ics-2026-bomblab$ ./defuser --seed 123456789 --phase 4 | ./bomb --seed 123456789 --phase 4
```

```
BOMBLAB 2026                                        这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

--------------------------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x1a1d587cd12d2d6b
Input (unsigned decimal integers):
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

```
lenovo_unix@SYLVIA-LAPTOP:~/ics-2026-bomblab$ ./defuser --seed 123456789 --phase 5 | ./bomb --seed 123456789 --phase 5
```

```
BOMBLAB 2026                                        这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

--------------------------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xb34f7324f11d12de
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

```
lenovo_unix@SYLVIA-LAPTOP:~/ics-2026-bomblab$ ./defuser --seed 123456789 --phase 6 | ./bomb --seed 123456789 --phase 6
```

```
BOMBLAB 2026                                        这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP    \
                    |               |
                     \             /
                      '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

--------------------------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0xd5c55b979d86d5c2
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

```
lenovo_unix@SYLVIA-LAPTOP:~/ics-2026-bomblab$ s=123456789
{ ./defuser --seed "$s"; ./defuser --secret --seed "$s"; } | ./bomb --seed "$s"
```

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

  许遥：

  北方那边的通知到了，我准备去。
  那个项目我想试很久了。你帮我看申请的时候，我还说不一定选得上，
  现在真能去了，反而有点不习惯。以后遇到问题，
  不能再把椅子滑到你旁边，问一句‘你看这是不是有毛病’。

  你想留下继续手里的事，我记得你为它准备了多久。
  我也想看看，最后能做成什么样。
  这里没做完的几处，我把记录整理好了。
  你有空就改，没空先忙自己的事。

  走之前出去吃顿饭吧。这次别在实验室凑合了。

  —— sky

隔了几天，许遥在后面添了一份回复。

  吃饭的时间你定。

  这里的东西我会留着。你在北方有了新想法，也可以回来添。
  我暂时不走，这边的事刚有起色，想再试一阵。
  以后说不定会改主意，到时候告诉你。你也一样。
  别因为申请时说得很坚定，遇到不喜欢的事就不好意思讲。

  第一场雪记得拍。我想看看你一直说的北方。

  —— 许遥

后面是这些年断断续续的更新。
一张北方的雪照，一次换了方向的项目，一段拖了很久才修好的代码。
sky 去了北方，许遥留在这里。他们各自忙起来，
回复有时隔了好几天，也会聊新认识的朋友。
有一阵，许遥没再继续原来的项目。后来 sky 也换了工作。
这座城倒还在。谁回来，就顺手添一点东西。

观测台的灯是 sky 修的，两把椅子是许遥后来搬来的。
最后一张便签贴在灯座旁边：

  ‘下次回来，先坐一会儿。’

林夏拉开其中一把椅子。你们坐了一会儿，没有再翻记录。
街上亮起今天的新灯。

离开观测台时，你想起自己收藏了很久的一份项目介绍，
把它放进明天的待办。林夏也在自己的本子上记了几行。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

---

谢谢你远道而来，与我相遇。

晚安。

—— sky

---

# 辅助工具说明

本实验一开始有些无从下手，因此使用了一些 ai 辅助，主要用于理解实验题意、解释部分看不明白的汇编代码、检查并修正 `defuser.c` 的求解代码中的问题。但是没有直接使用 ai 生成解答，ai 的分析和代码修改都经过自己的理解、思考和检查。